

# 함수근사와 DQN

Q-테이블의 한계와 함수근사 · 타겟 네트워크 · 경험 재현과 CartPole

Machine Learning 2 · 9주차

한국과학영재학교 (KSA)

Chapter 9

# 이번 주차 개요

- 2013년 딥마인드, Q-테이블을 **신경망**으로 통째로 바꿔 Atari를 학습 — **DQN**(Deep Q-Network) (Mnih et al., 2015, Nature).
- 표(tabular) 강화학습의 전제 “상태-행동 쌍을 표에 하나하나 적는다”를 버린다.

## 세 차시의 흐름 — “한 가지씩 불안정 원인을 찾아 고친다”

**9.1** 표가 불가능한 이유와 함수근사의 핵심(일반화), Q-신경망의 손실함수

**9.2** 문제 1: 목표가 움직인다 → **타겟 네트워크**

**9.3** 문제 2: 데이터가 상관된다 → **경험 재현** + CartPole 학습곡선

이전: Ch.8 캡스톤(표 기반 이론 전체). 다음: Ch.10 — 행동이 연속량이면  $\max_{a'}$ 를 못 취하므로 **정책기반 RL**로 건너든다.

---

## 9.1 Q-테이블의 한계와 함수근사

---

## Q-테이블이 감당 못 하는 규모 — 숫자로 셈한다

- Ch. 6 Q-learning:  $Q[s][a]$  표에 모든 상태-행동 쌍 저장.
- Atari 화면이 만들 수 있는 상태는 사실상 무한 — 답을 수도, 방문하기도 어렵다.
- DQN 논문이 실제로 쓴 화면:  $84 \times 84$  흑백, 픽셀당 256가지 밝기.

$$256^{84 \times 84} = 256^{7,056} \approx 10^{16,993}$$

- 바둑판  $10^{172}$  보다  $10^{16,800}$ 배 더 큰 상태 공간.
- 관측 가능 우주의 원자 수( $10^{80}$ )는 말할 나위도 없다.

**Q-테이블의 문제는 “메모리가 부족”이 아니라 원칙적으로 불가능하다.**

## 더 절망적인 문제: 규모가 아니라 “방문”

- 표가 담을 수 있다 해도, 학습은 **만난 상태**만 갱신한다.
- 100만 에피소드를 돌려도 Atari에서 실제로 보이는 화면은 극히 일부.
- 대다수 표 칸은 **영원히 0(초기값)**으로 남는다.

### “빈 칸”의 의미

칸이 비어 있다 = **그 상태의 Q값을 모름**. 비어 있는 칸이 나온 순간, 에이전트는 **어림도 없는 숫자**를 근거로 행동한다.

Ch. 4.3에서 짚은 “바둑처럼 상태가 많으면 표 저장 자체가 불가능” 문제가, 여기서 실제로 닥친다.

## 연속 상태: 차원의 저주

- 표 기반이라 해도 상태에 **연속량**이 섞이면 같은 문제.
- CartPole 상태  $(x, \dot{x}, \theta, \dot{\theta}) = 4$ 개의 실수.

| 좌표별 눈금  | 표의 칸 수                    |
|---------|---------------------------|
| 12등분    | $12^4 = 20,736$ — 아직 견딜 만 |
| 100등분   | $100^4 = 10^8$            |
| 1,000등분 | $10^{12}$                 |

(curse of dimensionality)

상태 공간이 눈금의 **4제곱**(차원수만큼 거듭제곱)으로 커진다. 연속 상태에서는 **어떤 눈금도** “표로 충분”하지 않다.

## 핵심 아이디어: 함수근사와 일반화

- $Q^*$ 는 상태-행동마다 다른 값을 주는 함수 — 표 대신 **수식으로** 표현할 함수족  $Q(s, a; \theta)$  ( $\theta$  = 가중치)로 근사: **함수근사**(function approximation).
- 표: “같은 칸” 상태끼리만 정보 공유. 다른 칸은 아무리 비슷해도 배운 것을 **아무것도** 전달하지 못한다.

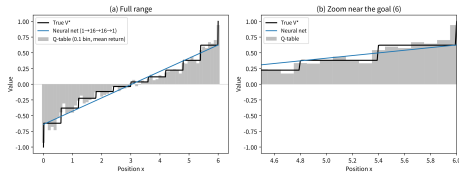
### (generalization)

함수가 매끄러우면, **한 번 본 상태**에서 배운 것이 **한 번도 본 적 없는 비슷한 상태**까지 자동으로 퍼져나간다. ML1의 “학습 데이터에 없었던 새 입력에 대한 예측”과 같은 개념.

**함수근사의 진짜 이유는 메모리가 아니라 일반화다.**

# 같은 데이터, 두 근사기: 표 vs 신경망

- 1차원 랜덤워크 장난감:  $x \in [0, 6]$ , 매 스텝  $\pm 0.6$  균일, 0에서  $-1$ , 6에서  $+1$  종료,  $\gamma = 0.9$ .
- **동일한** 300개 에피소드(시드 7)로 각각 학습.
- 표: 0.1칸(61칸) 평균 리턴 / 신경망:  $1 \rightarrow 16 \rightarrow 16 \rightarrow 1$  MLP.
- **본 적 있는 곳:** 둘 다 참값  $V^*$ 에 가까움(표의 미덕).
- **칸과 칸 사이:** 표는 보간이 없는데, 신경망은 매끄러운 곡선. **이 보간이 곧 일반화.**
- **표의 빈 칸:** 표는 답을 못 내는데, 신경망은 주변을 보고 추론.



(a) 전 구간: 표는 칸마다 점프, 신경망은 매끄러운 곡선. (b) 목표(6) 근처 확대: 표의 “칸”이 참값의 급격한 변화를 놓친다.

## 왜 일반화가 “장난감”이 아니라 필요 조건인가

- 장난감은 1차원이어서 “칸과 칸 사이”가 극적으로 드러난다.
- 실제 Atari 상태 공간(수천 차원)에서는 표의 “칸과 칸 사이”가 **전체 공간의 거의 100%**다.

### 한 줄 요약

표가 불가능한 “크기” 때문만은 아니다 — **만나지 못한 상태**에 대한 답도 내야 하기 때문에 함수로 가야 한다.

- **Q-테이블**: 본 칸만 정확, 나머지 공간은 답이 없음.
- **신경망**: 보지 못한 곳까지 보간.

## Q-함수를 신경망으로 근사: 손실함수

- DQN의 아이디어: 테이블 대신 **신경망**으로  $Q(s, a; \theta)$  근사.
- 손실함수 = Ch. 6의 **TD 오차**를 **제공한 것**:

$$J(\theta) = \mathbb{E} \left[ \left( r + \gamma \max_{a'} Q(s', a'; \theta) - Q(s, a; \theta) \right)^2 \right]$$

- 경사하강법(역전파)으로  $\theta$  갱신 — **형태는** 지도학습 회귀와 같음.
- 그러나 “정답”인 목표값  $r + \gamma \max_{a'} Q(s', a'; \theta)$  자체가 지금 학습 중인  $\theta$ 로 계산된다.

## “같은 $\theta$ 가 두 자리에” — 목표가 움직인다

- ①  $Q(s, a; \theta)$  는 **예측**.
- ②  $\max_{a'} Q(s', a'; \theta)$  는 **목표의 원료**.
  - 둘 다 같은  $\theta$ 이므로,  $\theta$ 를 갱신하면 예측이 목표에 가까워지는 **동시에 목표 자체도 움직인다**.
  - **과녁이 쏠 때마다 도망가는 셈이다** — 진동, 발산의 원인.

### 9.2절의 연결

이 식을 “정답이 고정된 지도학습”으로 착각하면 안 되는 이유가 바로 이 둘째 항의  $\theta$ 다. 타겟 네트워크가 이 두 자리를  $\theta$ 와  $\theta^-$ 로 분리하는 것이 9.2절.

# QNetwork: DQN의 최소 구조

```
import torch
import torch.nn as nn

class QNetwork(nn.Module):
    def __init__(self, state_dim, n_actions):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 64), nn.ReLU(),
            nn.Linear(64, 64), nn.ReLU(),
            nn.Linear(64, n_actions),
        )

    def forward(self, x):
        return self.net(x)
```

CartPole: QNetwork(4, 2). Atari면 입력을  $84 \times 84 \times 1$  이미지에 CNN을 붙여 특징을 뽑은 뒤 이 구조.

## 구조를 눈으로 따라간다: 출력 $|\mathcal{A}|$ 개

- 입력: 상태  $s$  (CartPole 4원 벡터 / Atari는  $84 \times 84$  이미지).
- 두 은닉층(각 64 뉴런, ReLU) → **출력  $|\mathcal{A}|$ 개** — 각 행동을 하나의 출로(outlet)로.
- $a = 0$ 번 출력 =  $Q(s, 0; \theta)$ ,  $a = 1$ 번 출력 =  $Q(s, 1; \theta)$ .

### 왜 “한 네트워크 - 여러 출력”인가

**한 번의 순전파**로 모든 행동을 동시에 평가  $\Rightarrow \max_{a'} Q(s', a'; \theta)$  는 출력 벡터의 `.max()` 한 줄.  
**공유된 은닉층**이 “비슷한 상태를 비슷한 Q값으로” 만드는 **공통 특징 추출기** — 이 구조가 일반화의 핵심.

## 파라미터 수를 센다: 4,610개

$$\underbrace{(4 \times 64 + 64)}_{\text{층 1}} + \underbrace{(64 \times 64 + 64)}_{\text{층 2}} + \underbrace{(64 \times 2 + 2)}_{\text{출력층}} = 4,610$$

- **4,610개**의 숫자가 CartPole 4차원 상태 공간 **전체**에서 Q함수를 정의. 같은 환경 12등분 표는 20,736칸, 100등분은  $10^8$ 칸.
- “10억 칸의 표”를 “4,610개의 가중치”로 대체.

### 본질

**파라미터 수는 상태 공간의 크기와 무관하게**(입력 차원에만 선형으로) 결정된다.

참고: Atari 이미지 7,056원을 완전연결로 넣으면 455,938개 — 여전히 미미. 단, 실제 DQN은 **CNN을 앞단에 붙여** 특징 수백 개로 압축 (ML1 Ch. 10).

## 손으로 한 스텝: DQN 갱신을 숫자로 (1/2)

CartPole(gym.make("CartPole-v1"), 시드 0), 초기화 직후(torch.manual\_seed(0), 아직 아무것도 못 배운) 네트워크:

- 초기  $s_0 = [0.0137, -0.0230, -0.0459, -0.0483]$ , 행동  $a = 1$ (오른쪽).
- 1스텝 후:  $s_1 = [0.0132, 0.1727, -0.0469, -0.3552]$ ,  $r = 1$ ,  $\text{done} = 0$ .

- 1 예측:  $Q(s_0, 1; \theta_{\text{init}}) = 0.0020$
- 2 목표 원료:  $\max_{a'} Q(s_1, a'; \theta_{\text{init}}) = 0.1509$
- 3 목표값:  $1 + 0.99 \times 0.1509 = \mathbf{1.1494}$

Ch. 6의 규칙: 살아남으면 +1, 최대 리턴 500.

## 손으로 한 스텝: DQN 갱신을 숫자로 (2/2)

④ **TD 오차:**  $\delta = 1.1494 - 0.0020 = 1.1474$

⑤ **그래디언트:**  $\frac{\partial J}{\partial Q(s_0, 1)} = 2(Q - \text{target}) = -2.2948$

⑥ **Adam 한 스텝** ( $lr = 10^{-3}$ ):  $Q(s_0, 1) : 0.0020 \rightarrow 0.0260$ , 손실  $1.3165 \rightarrow 1.2620$

### 핵심

여섯 줄이 전부고, 각 줄이 Ch. 6 표 Q-learning의 해당 줄과 1:1 대응한다. DQN과 표가 다른 것은 목표값 계산식이 아니라 저장·갱신 메커니즘.

## 표 vs DQN: 같은 목표, 다른 갱신 속도

| 단계     | 표 Q-learning (Ch. 6)                           | DQN (위 숫자)                  |
|--------|------------------------------------------------|-----------------------------|
| 현재 값   | $Q(s, a)$ (표 칸) = 0.0020                       | $Q(s, a; \theta) = 0.0020$  |
| 목표값    | = 1.1494                                       | = 1.1494                    |
| 갱신     | $Q \leftarrow Q + \alpha(\text{target} - Q)$   | 경사하강(역전파)                   |
| 한 스텝 후 | 0.0020 $\rightarrow$ 0.1167 ( $\alpha = 0.1$ ) | 0.0020 $\rightarrow$ 0.0260 |

- 같은 현재값, 같은 목표값에서 출발. 표는 한 스텝에 **목표까지의 10%**를 채움 ( $0.0020 + 0.1 \times 1.1474$ ).
- 표:  $0.0020 \rightarrow 0.1167 \rightarrow 0.2200 \rightarrow \dots \rightarrow 1.1494$ 로 지수적 접근. DQN도 같은 고정점이지만 **속도와 방식**이 다름.

## 역사: 함수근사 RL은 DQN에서 처음이 아니다

- 1990년 **Tesauro**,  $TD(\lambda)$  + 신경망의 **TD-GAMMON** (Tesauro, 1995) — 백검몬을 사람 못지않게 두는 시스템. “표 대신 신경망으로 가치함수를 배운다”가 동작한다는 것을 25년 전에 증명.
- Sutton & Barto(1998) 저서가 “값함수 근사”를 RL의 정규 장으로 정리. 1990~2000년대: 선형 근사( $\phi(s, a)^T w$ ), RBF·커널 등.

그런데 30년간 비선형(신경망) FA RL은 실전에 거의 안 썼다

이유 = 지금 배운 바로 그 불안정성: 목표가 움직임(9.2), 데이터가 상관됨(9.3), 부트스트래핑이 자기 오차를 증폭(Ch. 3).

**DQN의 진짜 기여: 두 공학 장치(타겟 네트워크 + 경험 재현)를 조합해 불안정성을 사실상 제어한 것.**

## Atari에서 폭발한 이유 + 자주 하는 실수 ①②

Atari: 상태 공간은  $10^{16,993}$ 이지만 게임 규칙이라는 구조 덕분에 중요한 상태는 “공이 라켓 근처인가, 벽에 부딪히기 직전인가” 같은 몇십 가지 패턴으로 압축된다 — 일반화가 이 패턴을 잡아내는 기계다.

- **실수 ①: DQN의  $lr$ 과 표 Q-learning의  $\alpha$ 를 같은 스케일로 비교.** 표의  $\alpha = 0.1$ 은 **값 단위**(목표에 10% 접근), Adam( $lr=1e-3$ )은 **가중치 단위**(0.001 이동). “0.1로 올려보자”는 거의 항상 발산·폭주 → “목표 접근 속도”를 **측정**해서 맞춘다.
- **실수 ②: 출력 1개로  $Q(s, a)$  출력.** 입력에 행동을 넣으면  $\max_{a'}$  계산에  $|\mathcal{A}|$ 번 순전파 필요. Atari( $|\mathcal{A}| = 18$ )이면 **18배** 계산 차이 → **출력 수 = 행동 수**가 표준 구조의 이유.

## 자주 하는 실수 ③④

- **실수 ③: “비슷해 보이는 상태는 반드시 비슷한 Q값” 가정.** 일반화는 “비슷한 입력 → 비슷한 출력” 가정 위에 서 있다. Atari에서 **한 픽셀**이 바뀌면 점수가 완전히 달라지는 상태가 존재 (공이 라켓을 스치느냐 아니냐) — 함수가 **매끄럽지 않음**, 일반화가 실패할 수 있음. DQN이 “대부분 잘 된다”는 것은 실패가 **희박**해서이지 보장되어서가 아니다.
- **실수 ④: “초기값이 0이니까 처음엔 아무것도 모름” 착각.** 초기화 직후 네트워크는 이미 값을 가진다 —  $Q(s_0, 0) = 0.1368$ ,  $Q(s_0, 1) = 0.0020$ (무작위 초기화). 초기값은 0이 아니라 **무작위** → 시드마다 다른 Q값, DQN이 **어떤 행동을 먼저 탐험할지**를 이 무작위성이 결정.

## 자주 묻는 질문 (정리)

- **PCA·word2vec의 “압축”과 같은가?** 방향은 비슷(고차원 원본을 다루기 쉬운 형태로). PCA·word2vec은 구조 보존(비지도), DQN은 “Q값 정확히 예측”이라는 **지도** 목표.
- **함수근사 = 비표 기반?** 사실상 같다 — 각 상태-행동을 **별개 파라미터**(표 칸)로 가지는 것 vs **파라미터를 공유하는 함수**로 표현. 선형 회귀·트리·RBF도 비표 기반이고, DQN은 그중 **신경망**을 고른 것.
- **일반화 = 과적합의 정반대?** 맞다. 너무 작은 망 = **부족 적합**, 너무 큰 망 = 배운 상태에만 과잉 적합 → 보지 못한 상태에서 오히려 더 나빠짐.

---

## 9.2 DQN의 안정화 장치와 실습

---

# 두 불안정 원인: 이번 주차의 지도

## 문제 1: 목표가 움직인다

- “정답”인 목표값이 매 스텝 스스로 바뀜.
- 해결: **타겟 네트워크** (이 절).

## 문제 2: 데이터가 상관된다

- 연속 경험이 서로 비슷해서 미니배치 분산이 커짐.
- 해결: **경험 재현** (9.3절).

## 실험의 목표

“움직이는 목표”가 학습을 **실제로 얼마나** 변질시키는지 숫자와 그래프로 확인하고, 그 실험이 건져 올린 **세 번째 문제(과대추정)**의 실마리를 9.3절 Double DQN으로 이어갈 고리를 남긴다.

## 문제 1: 목표가 계속 움직인다

- $\theta$ 가 매 스텝 갱신  $\Rightarrow$  손실의 목표값도 매 스텝 바뀜.
- 지도학습은 정답  $y$ 가 절대 안 바뀌는데, 여기서 “정답”이 스스로를 쫓아다니는 셈.
- 결과: 학습이 불안정해짐 (진동, 발산).

해결책: (Target Network)

$Q$ 와 똑같은 구조의 두 번째 신경망  $Q(s, a; \theta^-)$ 를 두고, 목표값 계산에는 **타겟 네트워크만** 쓴다:

$$J(\theta) = \mathbb{E} \left[ \left( r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

$\theta^-$ 는 매 스텝 갱신 **안 되고**, 일정 주기(예: 1000스텝)마다  $\theta$ 의 현재값으로 **통째로 복사**. 그 사이 목표값은 고정  $\Rightarrow$  짧은 구간 동안은 “정답이 고정된” 지도학습 최적화.

## dqn\_loss: 두 자리를 분리해 읽자

```
def dqn_loss(Q_net, target_net, states, actions,
            rewards, next_states, done, gamma):
    q_pred = Q_net(states).gather(1, actions.unsqueeze(1)).squeeze(1)
    with torch.no_grad():
        q_next = target_net(next_states).max(dim=1).values
        target = rewards + gamma * q_next * (1 - done)
    return ((target - q_pred) ** 2).mean()
```

- $q\_pred = \text{예측 } Q(s, a; \theta)$ ,  $q\_next = \text{목표의 원료 } \max_{a'} Q(s', a'; \theta^-)$  — 9.1절의 “같은  $\theta$  두 자리”가  $\theta, \theta^-$ 로 분리된 바로 그 두 자리.

## torch.no\_grad()의 두 가지 일

- ① 목표값 계산에 계산도를 짓지 않으므로 **메모리·계산 절약**.
- ② **이 스텝의 최적화 문제에서 목표값은 상수** — 그래디언트가 타겟 가중치로 흘러가지 않아 backward()가 건드릴 수 있는 것은  $\theta$  쪽뿐.

### 이 “상수 취급”이 없으면

손실을 줄이려는 그래디언트가 “예측을 목표로 당기는” 것뿐 아니라 **“목표를 예측 쪽으로 당기는”** 경로도 만들어버린다 (목표는 미분 가능한 입력이므로) — 움직이는 과녁 문제가 **미분 구조**까지 새어 들어온다.

“미분을 끊는다”  $\neq$  “값을 동결한다” — 값 동결에는 **바뀌지 않는 파라미터  $\theta^-$** 가 필요하다.

# DQN의 실제 구조: 합성곱 신경망

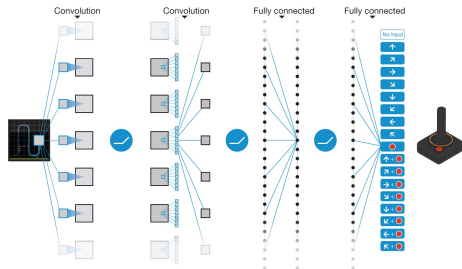
- Atari 화면을 입력받아 **조이스틱 행동별 Q값**을 출력.
- 앞단 CNN이 “화면 → 특징”을 압축한 뒤 완전연결로 Q값 예측 (ML1 Ch. 10의 CNN 재등장).

## 한 스텝 갱신을 숫자로: 미분 확인

목표값을 상수  $T$ 로 놓으면:

$$\frac{\partial L}{\partial Q(s, a; \theta)} = 2(Q(s, a; \theta) - T)$$

$T = r + \gamma \max_{a'} Q(s', a'; \theta^-)$  ( $\theta$ 에 대해 상수).  
그래디언트가 정확히 “**갭을 줄이는 방향**” — 회귀 손실  $(\hat{y} - y)^2$ 와 동일. 갱신의 **방향**이 이제 잘 정의된다.



원 논문 Extended Data Figure 1.

## 손으로 한 번: “움직이는 과녁”을 숫자로

- 극도로 단순한 장난감:  $Q(s, a; \theta) = \theta, r = 1, \gamma = 0.99$ .
- 학습 진행 중  $\theta : 0 \rightarrow 0.5 \rightarrow 0.9$ .

### 타겟 네트워크 없이

$$1 + 0.99 \times 0 = 1.0, \quad 1 + 0.99 \times 0.5 \approx 1.495, \quad 1 + 0.99 \times 0.9 \approx 1.891$$

환경·보상은 전혀 안 바뀌었는데, 목표값이 **1.0** → **1.495** → **1.891**로 계속 움직임 — “방금 도달한 목표”를 향해 갱신했더니 갱신 때문에 목표가 더 멀리 도망간 셈.

### 타겟 네트워크 있음

$\theta^-$ 가  $\theta_0 = 0$ 에 고정  $\Rightarrow$  두 번의 갱신 동안 목표값은 **정확히 1.0으로 그대로**. 이 숫자 하나로 “고정된 정답을 향해 안정 수렴”이 설명된다.

## 왜 “가까워질수록 더 멀어지는가”

- $1.0 \rightarrow 1.495 \rightarrow 1.891$ 은 **자기 가속** 구조.
- $\theta$ 가 목표에 가까워질수록  $\theta$ 가 커지고,  $\theta$ 가 커질수록 목표는 더 멀리 도망간다.
- 스텝 크기가 충분히 크면 이 **양(+)** 피드백이 **진동 또는 발산**으로 커진다.

### 초기 경험의 주된 원인

“DQN을 돌려 보면 가끔은 잘 되고, 가끔은 손실이 폭주한다”는 바로 이 메커니즘.

- 타겟 네트워크가 있는 경우:  $\theta^-$  동기화라는 “벽”이  $C$ 스텝마다 한 장씩 쳐지므로, 그 사이 목표는  $\theta$ 의 현재값과 **무관하게** 고정.

## 목표는 얼마나 “넓아도” 되는가 — $C$ 의 대가

- 하드 업데이트:  $C$ 스텝마다  $\theta$ 를 통째로 복사 (DQN 원논문 Atari 실험:  $C = 10,000$ ).
- 두 동기화 사이 목표는 동결 — 그만큼  $\theta$ 와  $\theta^-$ 가 최대 약  $C$ 스텝 벌어질 수 있음.

### 읽는 법

**목표는 고작  $C$ 스텝 “넓은” 값일 수 있다 — 안정성은 이 “느림”의 대가로 산 것이다.**

- 적절한  $C$ 는 환경 규모에 따라 다름: CartPole  $\approx 100$ , Atari급  $\approx 10,000$ .
- $\alpha, lr$ 과 마찬가지로 튜닝 대상 하이퍼파라미터.

## 정량화: “오차 수명” $1/(1 - \gamma)$

- **(1) 할인율 자체가 오차의 수명을 정한다.** 미래 오차는  $\gamma^k$ 만큼 할인되어 현재로 전파 — “신뢰할 수 있는 미래”의 길이는 사실상  $1/(1 - \gamma)$ 스텝.

| $\gamma$ | 오차 수명 $1/(1 - \gamma)$ | 오차 반감기   |
|----------|------------------------|----------|
| 0.9      | 10 스텝                  | 약 7 스텝   |
| 0.99     | 100 스텝                 | 약 69 스텝  |
| 0.999    | 1,000 스텝               | 약 693 스텝 |

- $\gamma = 0.99$ 이면 **100스텝 앞** 상태의 오차가 현재 목표에 거의 그대로 반영 — 목표가 “자신이 100스텝 전에 계산했을 때의 자신”을 따른다.
- **(2) 피드백 루프가 끊기지 않는다:** 오차  $\varepsilon \rightarrow$  목표  $\rightarrow$  추정  $\rightarrow$  오차, 매 스텝 **달혀** 있고 감쇠는  $(1 - \gamma)$ 에 비례( $\gamma = 0.99$ 면 한 스텝에 겨우 1%만 감쇠).

**타겟 네트워크는 이 루프를  $C$ 스텝 동안 물리적으로 자른다 — 목표는 과거 스냅샷( $\theta^-$ )이므로 현재 오차가 목표에 반영되는 통로가 없다.**

## 알고리즘 조립 ①: 초기화와 $\epsilon$ -greedy

```
def dqn_train(env, n_episodes, max_steps, batch_size=32, gamma=0.99,
              lr=1e-3, update_freq=100, eps_start=1.0, eps_min=0.05,
              eps_decay=0.995):
    Q_net = QNetwork(env.state_dim, env.n_actions)      # 의9.1 네트워크
    target_net = QNetwork(env.state_dim, env.n_actions)
    target_net.load_state_dict(Q_net.state_dict())      # 처음엔 둘이 동일
    optimizer = torch.optim.Adam(Q_net.parameters(), lr=lr)
    buffer = ReplayBuffer(capacity=10000)               # 의9.3 재현버퍼

    step_id = 0
    for ep in range(n_episodes):
        eps = max(eps_min, eps_start * eps_decay ** ep) # e-greedy, 감소
        s, done, total = env.reset(), False, 0.0
        for t in range(max_steps):
            if random.random() < eps:
                a = random.randrange(env.n_actions)      # 탐험
            else:
                with torch.no_grad():
                    a = int(Q_net(torch.tensor([s]))
                           .argmax(dim=1).item())        # 착취
            s2, r, done = env.step(a)
            buffer.push((s, a, r, s2, float(done)))
```

## 알고리즘 조립 ②: 학습 루프와 하드 동기화

```
if len(buffer) >= batch_size:      # 충분히차면학습시작
    batch = buffer.sample(batch_size)
    states = torch.tensor([b[0] for b in batch],
                           dtype=torch.float32)
    actions = torch.tensor([b[1] for b in batch])
    rewards = torch.tensor([b[2] for b in batch],
                             dtype=torch.float32)
    next_states = torch.tensor([b[3] for b in batch],
                                dtype=torch.float32)
    dones = torch.tensor([b[4] for b in batch],
                           dtype=torch.float32)
    loss = dqn_loss(Q_net, target_net, states, actions,
                    rewards, next_states, dones, gamma)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
if step_id % update_freq == 0:      # 하드업데이트 (C 스텝)
    target_net.load_state_dict(Q_net.state_dict())
s, step_id, total = s2, step_id + 1, total + r
if done: break
print(f"episode {ep + 1}: 리턴 {total:.0f}")
s = env.reset()
return Q_net
```

## 하이퍼파라미터를 하나씩 짚자

| 파라미터            | 의미                                                           |
|-----------------|--------------------------------------------------------------|
| lr=1e-3 + Adam  | DQN의 표준 시작점 (표의 $\alpha$ 와 단위 비교 실수는 9.1 참고)                 |
| update_freq=100 | 타겟 네트워크의 $C$ — 이 절의 주제                                       |
| eps_decay=0.995 | $\epsilon$ -greedy(Ch. 2)를 에피소드마다 0.995배 감소: “처음엔 탐색, 끝엔 착취” |
| batch_size=32   | RL에서는 작은 배치가 일반적 — 버퍼 샘플은 본래 “다른 정책 시대”의 경험을 섞고 있고 머         |
| capacity=10000  | 재현 버퍼 크기, 일반적으로 클수록 좋음 (9.3)                                 |

### 조립체가 보이는 순간

한 줄 한 줄이 어디서 왔는지 보면, DQN은 “새로운 알고리즘”이 아니라 **Q-learning(Ch. 6) + 함수근사(9.1) + 두 안정화 장치(9.2·9.3)**의 조립체다.

## 역사: “오래된 목표”는 DQN보다 오래된 아이디어

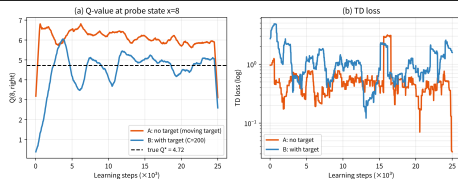
- 표 기반 Q-learning 갱신식에서 우변의  $Q(s', a')$ 은 **갱신 전** 값 — target을 먼저 계산하고 나서야  $Q[s][a]$ 를 쓰는 그 순서 하나. 표에서는 이 규칙이 **보이지 않는다**(각 칸이 독립 파라미터라서).
- 신경망에서는 모든 칸이 **같은  $\theta$ 를 공유**  $\Rightarrow$  “읽는 시점 vs 갱신 시점”의 차이는 **함수 전체의 차이**.  $\theta/\theta^-$  분리하는 표의 이 한 줄 규칙을 **독립 파라미터로 명시적으로 쓴 것**.
- 1990년 **TD-GAMMON**이 먼저 다뤘다 — “목표 계산용 네트워크를 덜 자주 갱신” 완화책. Sutton & Barto(1998): “지연된(damped) 목표”.
- 하드 업데이트 = 표의 “갱신 전 값” 규칙의 확장. **소프트 업데이트**  $\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^- =$  1960년대 확률근사 문헌의 **Polyak averaging**.
- 2018년 **Rainbow** (Hessel et al.)이 두 방식을 스펙트럼 하나로 통합; 연속제어 DDPG 계열에서는  $\tau = 10^{-3} \sim 10^{-2}$  소프트가 사실상 표준.

## 실습 설계: 통제된 대조 실험

- 타겟 네트워크 효과를 **격리**하기 위한 의도적 최소 설정: 경험 재현 **없이**(온-폴리시, 매 스텝 학습), 시드 1개, 작은 환경.
- **환경**: 1차원 랜덤워크. 상태  $x \in \{0, 1, \dots, 12\}$ , 좌/우 이동(성공 0.9, 미끄러짐 0.1), 매 스텝  $-1$ ,  $x = 12$  도달:  $+10$  종료(목표),  $x = 0$ :  $-10$  종료(위험),  $\gamma = 0.99$ . 탐침 상태  $x = 8$ (목표 근처, Q 차이가 커서 편향이 선명).
- **참값**: 값반복(value iteration, 5,000회 반복으로 수렴) —  $Q^*(8, \text{오른쪽}) = 4.720$ ,  $Q^*(8, \text{왼쪽}) = 2.642$ , 이점  $\approx 2.08$ .
- **두 팔**:  $1 \rightarrow 16 \rightarrow 16 \rightarrow 2$ , Adam  $lr = 10^{-2}$ ,  $\varepsilon = 0.1$  고정, 25,000스텝. (A) 타겟 **없음** vs (B) 타겟 **있음**(하드,  $C = 200$ ).

# 실험 결과: +27% 과대추정 vs +5%

|                        | 최종 $Q(8, \text{오른쪽})$ | 참값(4.720) 대비 | TD 손실 |
|------------------------|-----------------------|--------------|-------|
| A. 타겟 없음               | 6.012                 | +27% 과대추정    | 0.46  |
| B. 타겟 있음 ( $C = 200$ ) | 4.961                 | +5%          | 1.32  |
| 참값 $Q^*$               | 4.720                 | —            | —     |



(마지막 100샘플 평균. 시드 0. 13상태 환경이라 **크기**(27% vs 5%)는 이 실험의 수치일 뿐, **방향**은 견고하다.)

(a)  $x = 8$ 의  $Q$ (오른쪽). (b) TD 손실(로그 스케일).

## 그림에서 배우는 세 가지 ①②

- ① 움직이는 목표는 “떨림”이 아니라 “값을 잘못된 곳에 세운다”. (A)는 발산하지도 폭주하지도 않는다 — 손실은 꾸준히 떨어지고 Q값은 **약 6.0이라는 잘못된 값**에 안정. 참값보다 27% 높음. 발산보다 더 위험: 로그는 전부 정상인데, 에이전트가 “잘 learned된 거짓 믿음에 따라 행동.
- ② 왜 (A)는 과대추정인가 — **max의 구조적 편향**. 목표  $\max_{a'} Q(s', a')$ 는 추정치들 중 최대치를 고르는 연산. 오차가 섞이면 max는 오차가 크게 튀어 오른 쪽으로 기울어진다.

$$\mathbb{E}[\max(Q_A, Q_B)] = 5.0 + \frac{1}{6} \approx 5.167 \quad (\text{참 Q가 둘 다 5.0})$$

(선택 편향  $\approx +0.17$ .) 이 편향은 Q-learning 갱신에 **내장**되어 있고, 부트스트래핑이 스텝마다 재반영하며 증폭. 타겟 네트워크는 편향을 없애지 못하지만 **자기 증폭 루프를 차단** — 편향의 **근원**을 겨냥한 것이 **Double DQN**(9.3 예고: “행동은 온라인이 고르고, 평가는 타겟이 한다”).

## 그림에서 배우는 세 가지 ③ + 소프트 업데이트의 “느림”

- ③ (B)의 TD 손실이 0이 안 되고, 오히려 (A)보다 높다(1.32 vs 0.46). 낮은 목표 = 현재 관점에서 목표가 항상 약간 어긋남  $\Rightarrow$  학습 후에도 TD 오차 잔여가 남는다. 낮은 TD 손실  $\neq$  올바른 Q — 손실은 “자기 목표와의 일관성”이지 “참값과의 거리”가 아니다. 손실만으로 비교했다면 잘못된 쪽을 골랐을 것이다. 판정 기준은 항상 평가 리턴.

| $\tau$ | 효과적 지연 $1/\tau$ | 반감기      | 200스텝 후 남은 갭 |
|--------|-----------------|----------|--------------|
| 0.005  | 200 스텝          | 약 138 스텝 | 0.367        |
| 0.01   | 100 스텝          | 약 69 스텝  | 0.135        |

소프트 업데이트:  $g_{t+1} = (1 - \tau)g_t$ . 소프트  $\tau = 0.005 \approx$  하드  $C = 200$ . 이점은 연속성 — 하드처럼 목표가 C스텝마다 급격히 점프하지 않고 손실면이 매끄럽게 변함(비용: 매 스텝 목표용 순전파 1회).

## 자주 하는 실수

- **target\_net = Q\_net으로 “복사”**. 별칭(alias)일 뿐 값 복사가 아님 —  $\theta$ 가 갱신되면 타겟도 **자동으로** 갱신되므로 = 타겟 네트워크가 없는 것과 동일. load\_state\_dict는 값을 **별개** 파라미터로 복사. (확인: target\_net is Q\_net이 False여야 함.)
- **타겟 네트워크를 옵티마이저에 포함**. Adam(Q\_net.parameters() + target\_net.parameters())이면 타겟이 **조용히 학습**됨  $\Rightarrow$  “목표 고정” 전제가 무너짐.
- **torch.no\_grad() 빼먹기**. 계산도 두 배(메모리), “목표도 함께 당겨지는” 경로가 생김.

## 자주 하는 실수 (이어) + FAQ

- **update\_freq( $C$ )를 1로, 또는 동기화 자체를 안 하는 것.**  $C = 1$ 이면 매 스텝이 “동기화 직후” = 움직이는 목표 그 자체. 한 번도 하면 목표가 초기값으로 영원히 고정 — 아무것도 모르는 과녁을 쏘는 셈.  $C$ 는 “너무 낡아 지형이 바뀐 곳인가” vs “너무 새서 다시 움직이는가” 사이.
- **TD 손실을 학습 진행의 척도로 쓰는 것.** 진척도 판단은 평가 리턴(탐색 끄고 그리디 플레이)으로.

**FAQ:** 타겟은 학습되지 않지만 과거 스냅샷이라 “약간 낡았지만 안정적인 목표”. 벨만 연산자의 고정점( $Q^*$ )은 그대로 — 타겟은 경로만 바꾼다. 네트워크를 통째로 복사해야 “함수 전체의 스냅샷”이 됨(은닉층만 현재값이면 움직이는 목표가 은닉층에서 재등장). Adam 모멘텀·타겟(소프트)·BN running 통계량까지 세 가지 모두 “빠른 변수 + 느린 변수”로 급격한 변화를 부드럽게 하는 딥러닝 관용구.

---

## 9.3 경험 재현과 CartPole 학습곡선 실습

---

## 문제 2: 데이터 자체가 서로 상관돼 있다

- 연속된 프레임(상태)은 서로 거의 같음  $\Rightarrow$  순서대로 학습하면 **최근 몇 개의 비슷한 경험에 과도하게 치우친** 학습.

### 지도학습(ML1)

- 데이터셋을 **셔플**: 한 배치의 32개 샘플이 서로 무관하도록 섞음.

### 온-폴리시 강화학습

- 데이터 스트림은 **하나의 연속 궤적**.
- CartPole 연속 10프레임:  $\theta$ 가 스텝당 바뀔 수 있는 크기가 극히 작아  $s_t$ 와  $s_{t+1}$ 은 **거의 같은 벡터**.

**“배치 32개를 순서대로 가져온다” = 같은 한 궤적에서 32장을 연속 촬영한 사진을 한 묶음으로 주는 것과 같다.**

## 왜 “독립성”이 중요하나

- 미니배치 경사하강은  $\mathbb{E}[\nabla L]$ 을 **배치 평균**으로 추정. 추정이 좋으려면(분산이 작으려면) 배치 안 샘플이 **서로 독립**이어야 한다.
- 샘플이 거의 같으면 배치 평균은 **단 한 샘플의 경사**에 거의 의존  $\Rightarrow$  경사 추정치가 **극단적으로 높은 분산**.

(mixing time)

“32개를 모았다”는데 **효과적 표본수**는 32가 아니라 몇 개에 불과. 연속 데이터가 “몇 스텝을 건너뛰어야 무관해지는가”를 재는 것이 혼합시간. CartPole 막대 각도는 몇십 스텝에 걸쳐 천천히 바뀌므로 혼합시간도 **수십 스텝**  $\Rightarrow$  32개 연속 샘플은 **동일 관찰 1개의 32번 반복**에 가깝다.

# 해결책: 경험 재현(Experience Replay)

- 경험한  $(s, a, r, s')$ 를 바로 학습에 쓰지 않고, **재현 버퍼**(replay buffer)라는 큰 저장소에 쌓아둔다.
- 학습할 때 버퍼에서 **무작위**로 미니배치 샘플링.

## 이득 1: 상관 제거

- 버퍼 = “지난 10,000스텝”의 경험 혼합.
- 무작위 32개는 **시간적으로 멀리** 떨어져 있음 (혼합시간보다 훨씬 벌어진 묶음  $\Rightarrow$  사실상 독립).

## 이득 2: 데이터 재사용

- 한 번 모은 경험이 수천 번 샘플링되며 반복 학습에 사용.
- 지도학습의 “1 에포치” 데이터를 **수천 에포치 worth**로 재사용.

두 이득은 **서로 다른 문제**를 해결 — 분리해 명심. (Ch. 5.3의 “과거 경험을 다른 목적으로 재사용”과 같은 뿌리.)

# ReplayBuffer: 두 줄이 전부다

```
import random

class ReplayBuffer:
    def __init__(self, capacity):
        self.buffer = []
        self.capacity = capacity

    def push(self, transition):
        if len(self.buffer) >= self.capacity:
            self.buffer.pop(0) # 가장오래된항목밀려남
        self.buffer.append(transition)

    def sample(self, batch_size):
        return random.sample(self.buffer, batch_size) # 비복원추출
```

- push: **FIFO** 큐 — capacity에 닿으면 가장 오래된 항목(pop(0))이 밀려남.
- sample: random.sample **비복원 추출** (한 배치 안에서 같은 경험 두 번 금지).

참고: list.pop(0)은  $O(n)$  — 실제 구현은 collections.deque(maxlen=10000)로  $O(1)$ .

## 손으로 한 번: 버퍼가 차고, 샘플이 섞이는 것

| push 순서 | 버퍼 내용 (왼쪽 = 가장 오래됨)                  |
|---------|--------------------------------------|
| e1      | [e1]                                 |
| e2      | [e1, e2]                             |
| e3      | [e1, e2, e3]                         |
| e4      | [e1, e2, e3, e4]                     |
| e5      | [e2, e3, e4, e5] $\leftarrow$ e1 밀려남 |

- e2..e5를 복원추출 12번(시드 42):  $e2 \times 7$ ,  $e3 \times 3$ ,  $e4 \times 1$ ,  $e5 \times 1$  — **각 경험의 빈도가 무작위로 분포.**
- 결정적인 점: “어제 모은” e2와 “방금 모은” e5가 **같은 배치에 섞일 수 있다** — 이 **시간적 간격**이 상관 제거의 핵심.

## “무작위 균일” 그 이상: 우선순위 경험 재현

- 기본 sample은 모든 경험에 **같은 확률**.
- 그러나 “막대가 거의 쓰러진 순간”, “드문 상태의 큰 보상” 같은 **정보량이 큰 경험**이 더 많이 쓰이면 좋겠다는 직감.

(PER) (Schaul et al., 2016)

각 경험에 **TD 오차의 크기**에 비례한 우선순위를 붙여 **중요한 경험을 더 자주** 샘플링. 기본 DQN의 균일 샘플링 = 우선순위가 전부 1인 **특수한 경우**.

- (FAQ) 복원 random.choice로 뽑으면 안 되나? — 작동은 함. 비복원은 배치 내 중복이 없으므로 분산이 약간 더 큼. 버퍼가  $10^4$  이상이면 두 방식의 차이는 **무의미**하게 작다. 핵심은 “버퍼에서 **무작위**로 뽑는다”는 것.

# 버퍼 크기: 클수록 좋은 것도, 작은 게 좋은 것도 아니다

- capacity는 **메모리와 다양성**의 절충.

## 너무 작으면(예: 100)

- 버퍼가 금방 “돌아” 새 경험이 오래된 것을 **연속으로** 밀어냄.
- 버퍼 안에는 최근  $\approx$  capacity 스텝만  $\Rightarrow$  혼합시간에 비하면 여전히 **상관 강함**. 첫 이득이 반쯤 사라짐.

## 너무 크면(예: 1,000,000)

- 메모리 상승.
- 버퍼가 **학습 초반의 무작위 정책** 경험으로 가득 차 그 초기 분포가 지배  $\Rightarrow$  학습이 **느려질** 수 있음.

실전:  $10^4 \sim 10^6$ . CartPole은  $10^4$ , Atari는  $10^6$  이상. 9.2의 update\_freq와 같은 종류 — **하이퍼파라미터**.

## off-policy라서 가능한 일: “낡은” 경험의 재사용

- 버퍼의 경험은 **행동 정책**  $\mu$  (그 당시의  $\epsilon$ -greedy)로 모았다 — 학습이 진행되면 정책은 바뀌므로, 버퍼의 절반 이상은 “지금은 더 이상 그렇게 행동하지 않을” 정책의 산물.

### 왜 DQN은 이 낡은 경험을 그대로 써도 되는가

Q-learning은 **off-policy** (Ch. 6.3) — “데이터를 **누가** (어떤 정책으로) 모았는지”와 무관하게, **목표 정책** (greedy)의 Q값을 학습. 갱신식은  $\mu$ 에 의존하지 않는다.

- 반대로 **on-policy** (SARSA, Actor-Critic 상당수)는 “**지금 정책 자체**의 가치”를 배워야  $\Rightarrow$  과거 정책의 경험을 그대로 쓰면 이론적 보장이 **깨진다**.

**한 줄 요약: 경험 재현은 off-policy의 “누가 모았든 재사용” 자유를, “데이터 효율 + 상관 제거” 두 이득으로 교환하는 장치다.**

## 두 장치가 한 루프에서 만나는 자리: train\_step

```
# CartPole 기준 (state_dim=4, n_actions=2)
Q_net = QNetwork(4, 2)
target_net = QNetwork(4, 2)
target_net.load_state_dict(Q_net.state_dict()) # 처음엔 동일
optimizer = torch.optim.Adam(Q_net.parameters(), lr=1e-3)
buffer = ReplayBuffer(capacity=10000)

def train_step(batch_size, gamma):
    batch = buffer.sample(batch_size) # 의9.3 재현
    states = torch.tensor([b[0] for b in batch], dtype=torch.float32)
    actions = torch.tensor([b[1] for b in batch])
    rewards = torch.tensor([b[2] for b in batch], dtype=torch.float32)
    next_states = torch.tensor([b[3] for b in batch], dtype=torch.float32)
    dones = torch.tensor([b[4] for b in batch], dtype=torch.float32)
    loss = dqn_loss(Q_net, target_net, states, actions, rewards,
                   next_states, dones, gamma) # 의9.2 타겟네트워크
    optimizer.zero_grad()
    loss.backward() # 의9.1 역전파
    optimizer.step()
    return loss.item()
```

## train\_step을 한 줄 한 줄 대응해 읽자

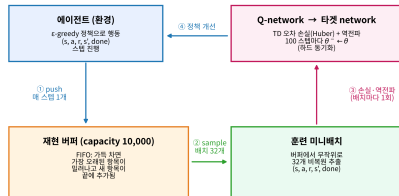
- `batch = buffer.sample(batch_size)` — **경험 재현이 들어오는 유일한 줄**. 32개는 지난 10,000스텝에서 무작위 비복원 추출  $\Rightarrow$  시간적으로 멀리 떨어져, 독립에 가까움.
- 나머지: 9.1의 QNetwork로 텐서 조립  $\rightarrow$  9.2의 `dqn_loss`가 **타겟 네트워크**  $Q(s', a'; \theta^-)$ 로 목표 계산  $\rightarrow$  `optimizer.step()`로  $\theta$ **만** 갱신 (타겟은 `no_grad()`로 상수).

한 스텝 = “버퍼 샘플링(9.3)  $\rightarrow$  목표값 계산(9.2)  $\rightarrow$  역전파(9.1)”의 한 바퀴

행동 선택( $\epsilon$ -greedy)은 **현재** 네트워크의 출력을 쓰고, 그 결과를 버퍼에 push한 **다음**에 샘플링해 학습. 행동은 “**지금 정책**”이 하되, 학습은 “**과거 + 지금**”의 혼합 경험을 한다 — 이 **비대칭**이 off-policy의 본체.

# 재현 버퍼가 루프 안에 끼어드는 자리

- 1 에이전트가 매 스텝 1개의  $(s, a, r, s', \text{done})$ 를 버퍼에 push(①).
- 2 버퍼가 1,000개(워밍업) 차면, 매 스텝 무작위 32개 미니배치 추출(②).
- 3 Q-net → 타겟-net의 **TD 손실** + **역전파**(③)로  $\theta$  갱신.
- 4 100스텝마다  $\theta^-$ 를  $\theta$ 로 **하드 동기화**(④).



① ② ③ ④가 학습 루프의 한 바퀴 — ①은 매 스텝마다, ②③은 버퍼에 1,000개(워밍업)가 쌓인 뒤 매 스텝, ④는 100스텝 주기로.

# CartPole에서 DQN 실습: 설정

- **300 에피소드** 실행. 9.2의 dqn\_train과 같은 설정:
- QNetwork(4,2), Adam  $lr = 10^{-3}$ , batch\_size=32,  $\gamma = 0.99$ .
- **Huber loss** — TD 오차 절댓값이 클 때 MSE보다 안정 (큰 오차의 기울기 폭주 방지).
- update\_freq=100(타겟 하드 동기화),  $\epsilon$ : 1.0에서 0.05까지 **스텝당 0.9997배** 감쇠.
- capacity=10000, **워밍업 1,000스텝** (버퍼 1,000개 모일 때까지 학습 없이 경험만 수집).

CartPole의 최대 리턴은 **500**

500스텝 버티면 **타임아웃으로 만점.**

## 결과: 21.15 → 103.85, 만점 500 도달

처음 20 에피소드 평균 리턴: 21.15

마지막 20 에피소드 평균 리턴: 103.85

학습 중 도달한 최고 리턴: 500.0 (만점)

episode 0: 9

episode 50: 11

episode 100: 66

episode 150: 120

episode 200: 259

episode 250: 500

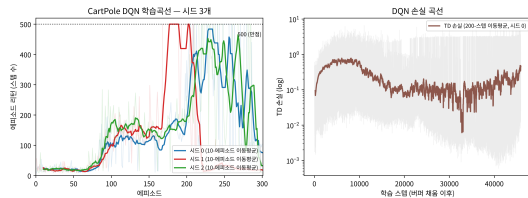
- 평균 리턴 약 **5배** 증가, 학습 중 최소 한 번 **만점 500**. Q-테이블 없이 **신경망 하나**가 “막대를 얼마나 오래 세울 수 있는지”를 스스로 익힘.
- 그러나 시드 0은 ep 200(259) → 250(500) → 299(104)로 **오르내리며 진동**. **시드 1의 마지막 20평균은 19** (시드 2: 141) — 시드마다 천차만별. 세 시드 모두 도중 만점 500에 도달.

# 학습곡선을 읽는 세 가지

① 곡선은 “계단”이 아니라 “들쭉날쭉한 상승”. 개별 에피소드는 5~500까지 튀어 다님. **이동평균**(두꺼운 선)이 진짜 진행 —  $\epsilon \approx 1$ (완전 무작위)에서 리턴 10~30로 시작해  $\epsilon$  감소와 함께 상승. 이 “들쭉날쭉” = 9.2의 불안정성 + 9.3의 불안정성이 **겹친 결과**.

② “**만점 500에 도달**”의 의미. 500은 **타임아웃** — 한 번 500을 찍었어도 “500을 안정적으로 낸다”는 뜻은 아님. “한 번 만점” vs “안정적으로 만점”을 구분하는 것이 평가의 핵심 (마지막 N 에피소드 평균, 또는  $\epsilon = 0$  그리디 N회 평균).

③ 손실 곡선이 “떨어진다”  $\neq$  “잘 배우고 있다”. TD 손실 하락 = “자기(타겟) 목표와 점점 일관적”일 뿐 (9.2 교훈: 낮은 TD 손실  $\neq$  올바른 Q). 진짜 판정 기준 = **평가 리턴**.



(a) 에피소드 리턴, 시드 3개. 검정 점선 = 만점 500.

(b) TD 손실(로그, 시드 0).

## 자주 하는 실수: 버퍼가 차기 전에 학습을 시작하는 것

- 학습 루프의 `if len(buffer) >= batch_size`: 조건을 빼먹고 버퍼가 거의 비어 있는 상태에서부터 학습  $\Rightarrow$  배치 샘플이 사실상 **같은(극소수) 경험의 반복 추출**  $\Rightarrow$  다양성 부족, 불안정한 학습.

### 왜 1,000개 이상인가

`batch_size=32`라면 “32개만 차면 학습”도 **가능**은 하지만, 32개 안의 경험은 **최근 32스텝**뿐 — 혼합시간(몇십 스텝)과 같은 길이이므로 **상관 제거 효과가 거의 없음**. 1,000개면 최근 1,000스텝이 섞여 32개 샘플이 시간적으로 **넓게** 퍼진다. **최소** 조건 = 버퍼 > 배치 크기, **실용** 조건 = 혼합시간보다 충분히 크 것.

- **연관 실수: ReplayBuffer 없이 순서대로 온-폴리시로만.** 9.2의 별칭 버그가 여기까지 오면, 문제 2(상관)가 **완전히** 회귀 — 발산하거나, 오래 진동하다 “**잘못된 값에 안정**”(9.2의 (A) 팔). 두 장치는 독립적으로도 필요하고, **함께 쓰일 때** 9.1의 “불안정한 조합”을 제어한다.

## 여러 시드: 한 곡선으로 판단하지 마라

- 위 숫자( $21.15 \rightarrow 103.85$ )는 **시드 0** 한 시드의 대표값.
- 신경망 RL은 **세 곳의 무작위성**을 동시에 안고 있음: 초기화,  $\varepsilon$ -탐색의 행동 선택, 버퍼 샘플링  
⇒ **시드마다 곡선 모양이 다름** (500 도달 시점, 이동평균 최종 높이).

### 표준: 여러 시도의 평균 곡선

한 시드만 보고 “안 된다/잘 된다”로 결론 = 한 주가의 주가만 보고 “나쁜 회사/좋은 회사”를 단정하는 것. **최소 3~5개 시드로 평균  $\pm$  표준편차**를 보고, (1) “이동평균이 **일관되게** 오르는가”(방향)와 (2) “시드 간 편차가 어느 정도인가”(안정성)를 **둘 다** 평가.

Ch. 8.1 팀 프로젝트 FAQ의 “한 실험으로 결론 내리지 않는다”의 강화학습 버전.

## 두 장치의 공통점

|                  | 지도학습의 전제                        | RL에서 왜 깨지나                                        | DQN의 장치                                                             |
|------------------|---------------------------------|---------------------------------------------------|---------------------------------------------------------------------|
| 데이터 독립성<br>정답 고정 | 배치가 i.i.d.<br>$y$ 가 학습 중 변하지 않음 | 연속 프레임이 서로 같음 (상관)<br>목표값이 매 스텝 $\theta$ 와 함께 움직임 | <b>경험 재현</b> (버퍼 무작위 샘플링)<br><b>타겟 네트워크</b> ( $\theta^-$ 를 $C$ 스텝 동 |

### 한 문장으로

“지도학습이 잘 작동하는 조건(독립 데이터, 고정 정답)을, 그 조건이 깨진 RL 환경에서 인위적으로 다시 만들어준다.” — 경험 재현은 데이터 자리, 타겟 네트워크는 정답 자리에 각각 지도학습의 조건을 되살린다.

“RL에 신경망을 붙이면 저절로 잘 될 것”이라는 순진한 기대는 틀렸다 — DQN의 진짜 기여는 신경망 자체가 아니라, 그 조합을 안정적으로 학습되게 만든 공학 장치다.

## 역사: “경험을 다시 쓴다”는 아이디어의 뿌리

- 경험 재현은 DQN이 처음 발명한 것이 아니다 — “과거 경험을 다시 쓴다”는 정신은 1990년대 표 기반 문헌에서 이미 여러 갈래:
- **Dyna**(Sutton, 1991, Ch. 7.3): 실제 경험과 **모델 시뮬레이션** 경험을 같은 스트림에 섞어 TD 갱신 (“여러 출처의 경험을 하나로 합쳐 재사용”의 원형).
- **$Q(\lambda)$** (Ch. 7.2): 적격 이력으로 한 경험을 **여러 스텝에 걸쳐** 재사용.
- **배치(오프라인) RL**: 모은 데이터를 **여러 에포크** 반복 사용.

### DQN = 규모의 승리

이 뿌리 위에 **신경망** + **대규모 버퍼**( $10^4 \sim 10^6$ ) + **버퍼에서 무작위 재샘플링**을 태워, Atari의 **수백만 프레임**에서 상관 문제를 “단순한 무작위 추출 한 줄”로 해결.

원논문 DQN: 버퍼  $10^6$  프레임, 배치 32, **4스텝마다 1회** 학습 — “4스텝마다”는 매 프레임 학습보다 상관(4프레임은 거의 같은 화면)을 **더 강하게** 끊으려는 추가 상관 제거.

## 이번 주차 정리

- ① **9.1 — 함수근사의 진짜 이유는 일반화** — Atari 상태 공간  $10^{16,993}$ 은 표로 원리적으로 불가. “4,610개 파라미터가 10억 칸을 대체”의 핵심은 **보지 못한 상태에 대한 보간**. 손실의  $\max_{a'} Q(s', a'; \theta)$  항이 “목표가 움직인다”의 근원.
- ② **9.2 — 타겟 네트워크** — 목표 계산 전용  $\theta^-$ 를 C스텝마다 하드 동기화해 “움직이는 과녁”을 고정. 실험: 타겟 없이는 **+27% 과대추정**로 “잘못된 값에 안정”. 낮은 TD 손실  $\neq$  올바른 Q.
- ③ **9.3 — 경험 재현** — 버퍼에서 무작위 미니배치로 **상관 제거 + 데이터 재사용**. off-policy가 낡은 경험 재사용을 허용. CartPole 21→104(시드 0), 세 시드 모두 만점 500 도달, **여러 시드의 평균 곡선**으로 판단.

### 다음 주 — Chapter 10. 정책기반 강화학습

DQN은 매 스텝  $\max_{a'} Q(s', a')$ 를 계산하는 구조. **행동이 연속량이면** 이 최댓값을 취하는 것 자체가 불가능  $\Rightarrow$  Q-함수를 우회하고 **정책을 직접 학습**하는 policy-based RL(REINFORCE)로 건너든다.